

DA5300: Data Structures For Data Science

Laboratory 3 – Linked Lists and Positional Lists

Implement a complete linked-list library in Python. The objective of this exercise is to understand the implementation of singly linked lists, doubly linked lists, and the Positional List ADT.

Part A: Singly Linked List

Implement a class `SinglyLinkedList` using nodes. Each node should store an element and a reference to the next node.

The class should support the following operations:

- (a) **Add at the beginning:** Insert a new element at the beginning of the list.
- (b) **Add at the end:** Insert a new element at the end of the list.
- (c) **Add unique element:** Insert an element only if the element is not already present in the list.
- (d) **Delete first occurrence:** Delete the first occurrence of a specified element from the list. If the element is not present, the list should remain unchanged.
- (e) **Delete all occurrences:** Delete every occurrence of a specified element from the list.
- (f) **Add after a given node:** Given a reference to a node, insert a new node immediately after it.
- (g) **Add before a given node:** Given a reference to a node, insert a new node immediately before it.
- (h) **Delete after a given node:** Delete the node immediately following a given node.
- (i) **Delete before a given node:** Delete the node immediately preceding a given node.

Your implementation should also provide suitable methods for displaying the elements of the list and determining whether the list is empty.

Part B: Doubly Linked List

Implement a class `DoublyLinkedList` using nodes containing three fields: an element, a reference to the previous node, and a reference to the next node.

Implement all the operations specified in Part A:

- (a) Add an element at the beginning.
- (b) Add an element at the end.
- (c) Add an element only if it is unique.
- (d) Delete the first occurrence of an element.
- (e) Delete all occurrences of an element.
- (f) Add a node after a given node.
- (g) Add a node before a given node.
- (h) Delete a node after a given node.
- (i) Delete a node before a given node.

In addition, provide methods to traverse the list both in the forward and reverse directions.

Part C: Positional List ADT

Extend your doubly linked list implementation to construct a `PositionalList`. A position represents the location of an element in the list and should provide access to the element stored at that position.

If p is a valid position, the following operation should be supported:

`p.element()` Return the element stored at position p .

Implement the following accessor methods for a positional list L :

- (a) `L.first()`: Return the position of the first element of L , or `None` if L is empty.
- (b) `L.last()`: Return the position of the last element of L , or `None` if L is empty.
- (c) `L.before(p)`: Return the position immediately before position p , or `None` if p is the first position.
- (d) `L.after(p)`: Return the position immediately after position p , or `None` if p is the last position.
- (e) `L.is_empty()`: Return `True` if L contains no elements and `False` otherwise.
- (f) `len(L)`: Return the number of elements in L .
- (g) `iter(L)`: Return an iterator that traverses the elements of L from the first position to the last position.

Implement the following update methods:

- (a) `L.add_first(e)`: Insert a new element e at the beginning of L and return the position of the new element.
- (b) `L.add_last(e)`: Insert a new element e at the end of L and return the position of the new element.
- (c) `L.add_before(p,e)`: Insert a new element e immediately before position p and return the position of the new element.
- (d) `L.add_after(p,e)`: Insert a new element e immediately after position p and return the position of the new element.
- (e) `L.delete(p)`: Remove the element at position p and return the removed element.
- (f) `L.replace(p,e)`: Replace the element stored at position p with e and return the previously stored element.

Requirements

1. Do not use Python's built-in `list` data structure to implement the linked lists.
2. Define appropriate `Node` classes for the singly and doubly linked lists.
3. For the positional list, ensure that a position cannot be used after the corresponding node has been deleted.
4. Handle special cases such as:
 - an empty list,
 - insertion into an empty list,
 - deletion of the first element,
 - deletion of the last element,
 - deletion of the only element,
 - insertion before or after the first/last position.
5. Write a test program demonstrating every operation implemented in Parts A, B, and C.
6. For each operation, state its worst-case time complexity in terms of n , where n is the number of elements in the list.

Suggested Testing

For each implementation, construct a list containing at least the following sequence of elements:

10, 20, 30, 20, 40, 50, 20.

Use this list to demonstrate insertion, deletion of one occurrence, deletion of all occurrences, insertion before and after a specified node/position, traversal, and the remaining Positional List operations.